DG Interns Hub

Cybersecurity Internship

Week 3 Internship Report

Linux Logs, Sysmon, auditd and Snort

Intern name	Harini E
Intern ID	DG/15/OCT/CYBER/040
Position	Cyber Security Intern
Tools used	Sysmon, auditd, Snort 3 and Nmap
Lab	VirtualBox: Windows 11, Kali Linux and Ubuntu 26.04 LTS

Contents

1. Summary . . . 3

2. Lab environment . . . 3

3. Theory notes . . . 3

4. Sysmon on Windows . . . 4

5. auditd on Ubuntu . . . 7

6. Snort . . . 8

7. Cheat sheet . . . 12

8. Rooms and courses completed . . . 12

9. Problems I faced and how I solved them . . . 13

10. What I learned . . . 13

11. Conclusion . . . 14

Appendix A: local.rules . . . 14

Appendix B: Week 3 folder structure . . . 14

1. Summary

Week 3 of my cybersecurity internship with DG Interns Hub was about endpoint and network detection. I worked with Sysmon on Windows, auditd on Ubuntu and Snort for network alerts, and I tested each of them in my own home lab. This report explains what I set up, what I tested, what went wrong along the way and how I fixed it, with screenshots as evidence.

The part that taught me the most was Snort. Ubuntu 26.04 has no Snort package, so I built Snort 3 from source, and then adjusted the task rules so that they worked in Snort 3. After that, an Nmap scan from my Kali machine triggered my own rule, mapped to MITRE ATT&CK T1046.

Week 3 at a glance

Task	Status
Theory notes (10 questions)	Completed, one file for each question
TryHackMe rooms (4)	Completed
LetsDefend courses (6)	Completed
Hands-on 1 and 2: Sysmon install and four events	Completed, events found in Event Viewer
Hands-on 3: auditd and ausearch	Completed, command found in the audit log
Hands-on 4: Snort setup and starter rules	Completed, Snort 3 built from source
Hands-on 5: Nmap SYN scan rule (MITRE T1046)	Completed, alerts captured
Hands-on 6: cheat sheet	Completed, 19 entries
LinkedIn posts (2)	Posted

2. Lab environment

All the work was done in a small VirtualBox lab on my laptop. The three machines were connected through a host-only network, 192.168.56.0/24.

Machine	Role in the lab	Address
Windows 11 laptop	Host machine, Sysmon and Event Viewer	192.168.56.1
Kali Linux VM	Attacker machine for the tests (ping, web request, SSH, Nmap)	192.168.56.101
Ubuntu 26.04 LTS VM (ubuntu-lab)	auditd and Snort, interface enp0s8	192.168.56.102

Tools used

• Sysmon v15.22 (Sysinternals) with the SwiftOnSecurity configuration file

• auditd 4.1.2 and ausearch on Ubuntu

• Snort 3.12.2.0, built from source

• Nmap 7.991 on Kali, Windows Event Viewer and PowerShell

• All tools used are free, as the task rules require

3. Theory notes

I wrote the ten theory answers in my own words, each in a separate file, and saved them in the Notes folder. The topics were:

No.	Topic
1	Endpoint Detection and Response (EDR), and how it differs from traditional antivirus
2	Sysmon, and how it differs from standard Windows Event Logs
3	auditd on Linux, and the difference between auditd and syslog
4	How Snort works, and Snort in IDS mode versus IPS mode
5	The structure of a Snort rule (header and options)
6	Signature-based versus anomaly-based detection
7	MITRE ATT&CK T1046 (Network Service Discovery) and mapping Snort rules to it
8	Comparison of Windows Event Log, Sysmon and Linux auditd
9	Key Linux log files a SOC analyst should monitor
10	Log ingestion, and the flow from a log source to a SIEM dashboard

4. Sysmon on Windows

4.1 Installation

I downloaded Sysmon from the Sysinternals site and the SwiftOnSecurity configuration file, and saved both in C:\Sysmon. I installed it from an elevated PowerShell window with the 64-bit file:

```
.\Sysmon64.exe -accepteula -i sysmonconfig-export.xml
```

My first attempt failed with the message "You need to launch Sysmon as an Administrator", because the window was not elevated. After I reopened PowerShell as Administrator, the install worked. I checked that the Sysmon64 service was running, and then opened Event Viewer to confirm that events were arriving in Applications and Services Logs > Microsoft > Windows > Sysmon > Operational.

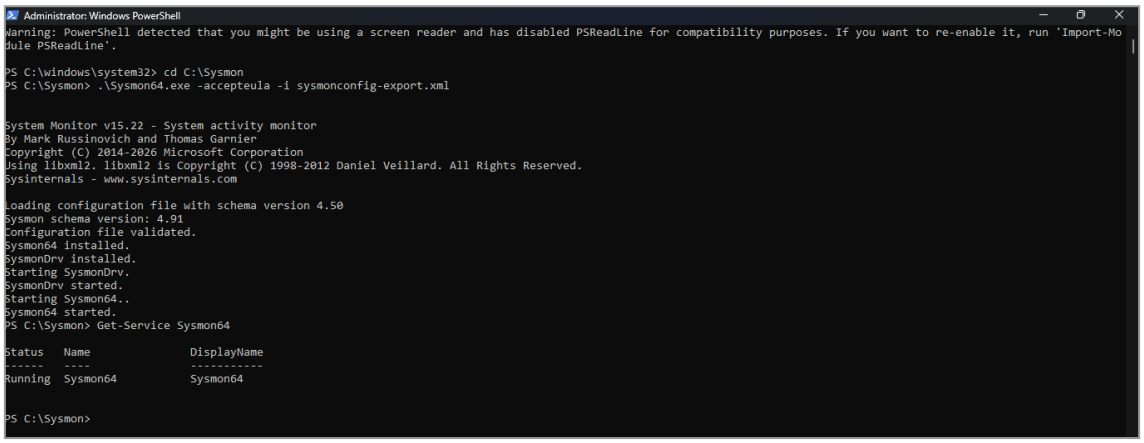


Figure 1: Sysmon64 and SysmonDrv installed and started, and the Sysmon64 service showing Running

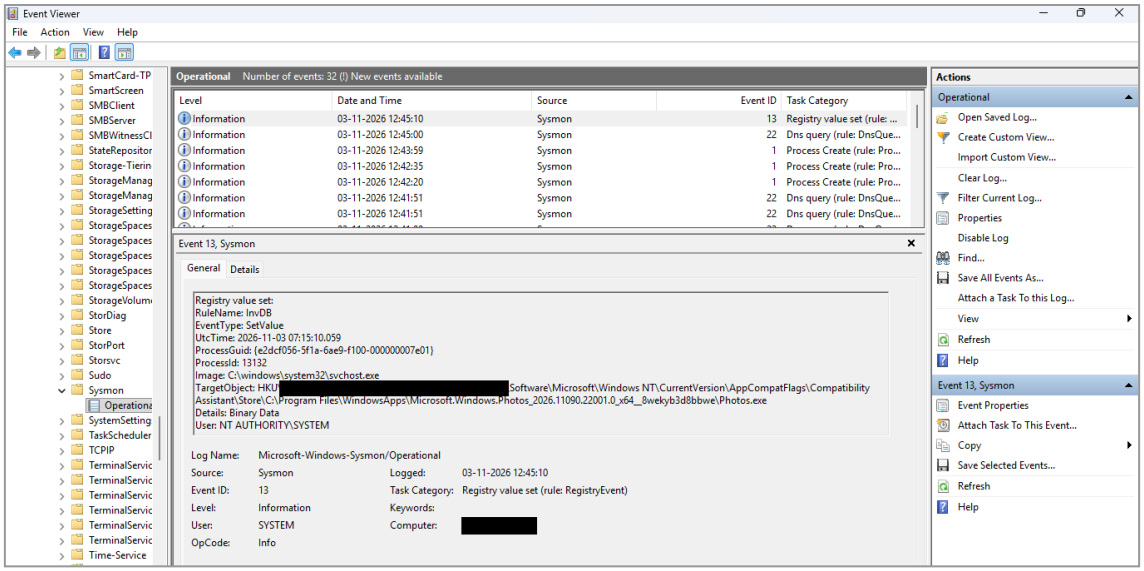


Figure 2: The Sysmon Operational log in Event Viewer showing events being captured (computer name hidden)

4.2 Making events and finding them

I then performed four actions and located the matching event for each one. The times below are local time on 3 November 2026.

Action	Event ID	Time	Key fields
Opened PowerShell	1 (Process Create)	12:52:15	Image: powershell.exe, its CommandLine and the current directory
Created a test user account	1 (Process Create)	12:52:35	Image: net.exe (then net1.exe), CommandLine: net user ... /add
Made a network connection	3 (Network Connection)	12:52:52	Image: powershell.exe, SourceIp 10.146.250.193, DestinationIp 104.20.23.154, port 443
Created a text file	11 (File Create)	12:56:46	Image: powershell.exe, TargetFilename: sysmon-test.txt in Downloads

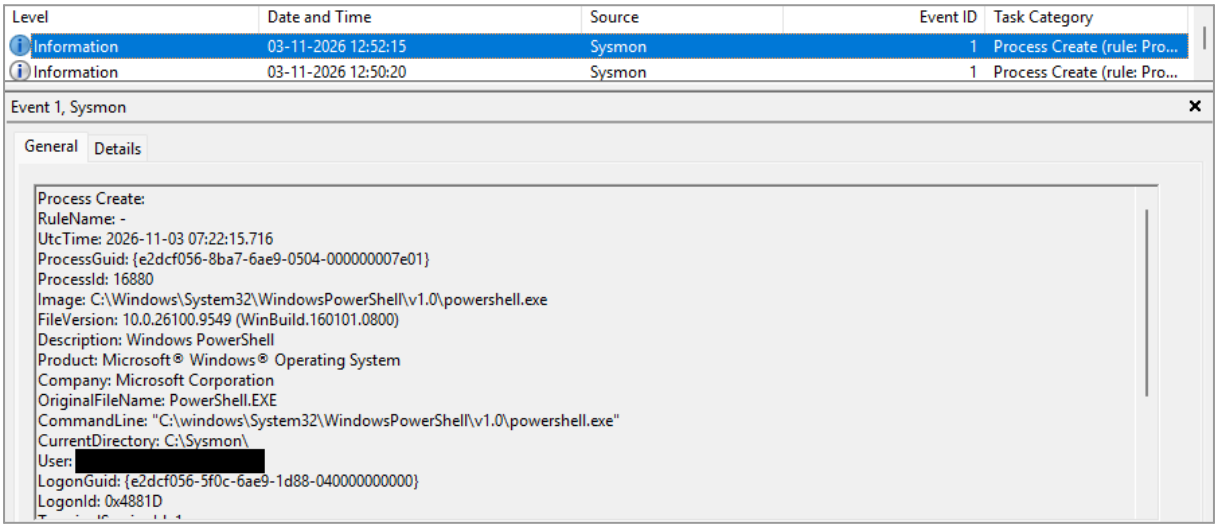


Figure 3: Event ID 1: opening PowerShell. The CommandLine and directory are recorded (username hidden)

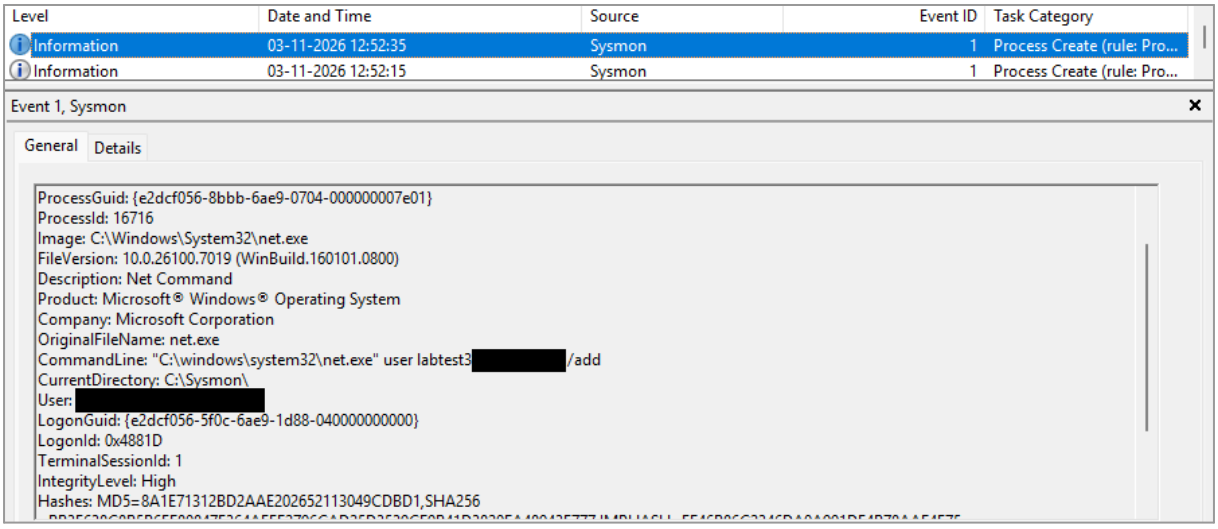


Figure 4: Event ID 1: the net user command used to create a test account (username and password hidden)

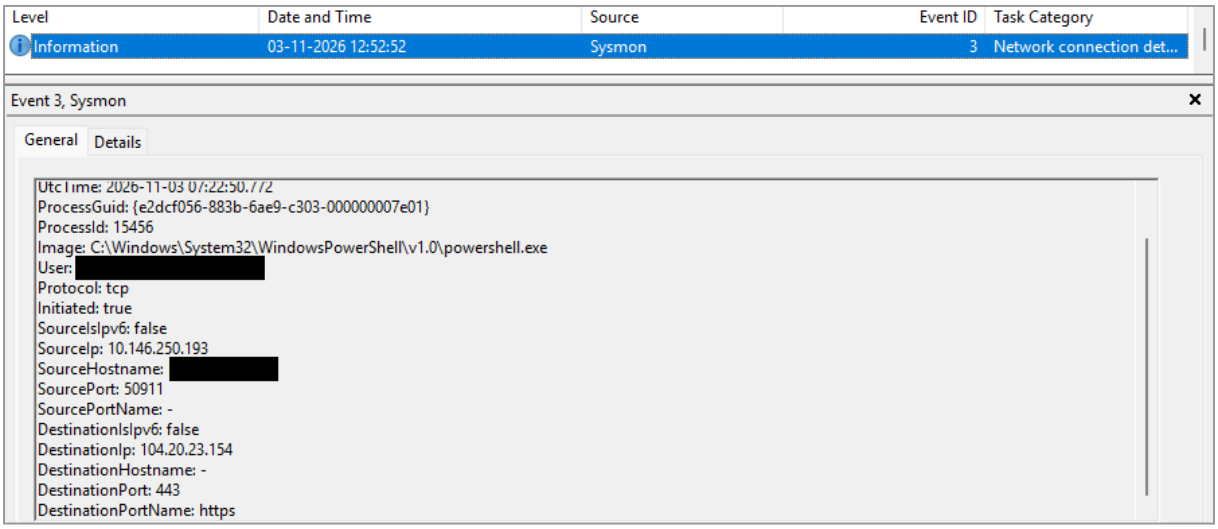


Figure 5: Event ID 3: powershell.exe connecting to 104.20.23.154 on port 443 (username and hostname hidden)

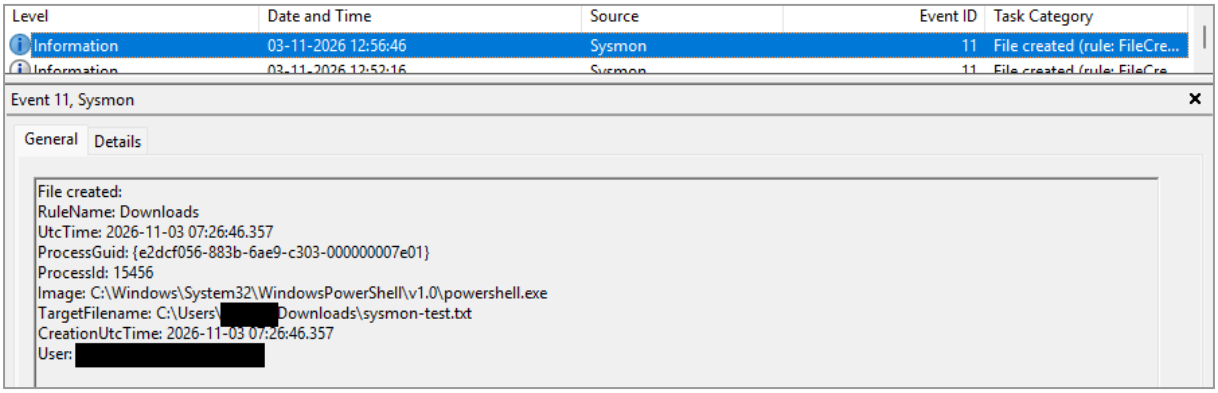


Figure 6: Event ID 11: powershell.exe creating sysmon-test.txt in the Downloads folder (username hidden)

4.3 What I noticed

• My first test file, created in the Temp folder, was not logged at all. The SwiftOnSecurity configuration only records file creation for certain folders and file types, so I created the file in Downloads and it appeared as Event ID 11. This reminded me to test what a configuration really covers.

• Running netstat -ano was also logged as its own Event ID 1 (NETSTAT.EXE).

• The dummy account and the test files were deleted after I took the screenshots.

A separate document, sysmon-events-documentation.pdf, explains each event in more detail and why it matters to a SOC analyst.

5. auditd on Ubuntu

5.1 Setup

On the Ubuntu VM I installed auditd, started it and enabled it with sudo systemctl enable auditd --now, and checked that the service was running. The task asks to search for the key "nano", but a key only exists if a rule creates it, so I added a rule that records every program run by apt:

```
sudo auditctl -a always,exit -F arch=b64 -S execve -F exe=/usr/bin/apt -k nano
```

I checked the rule with sudo auditctl -l, then ran sudo apt install nano. The package was already installed, but apt still ran, so the command was recorded.

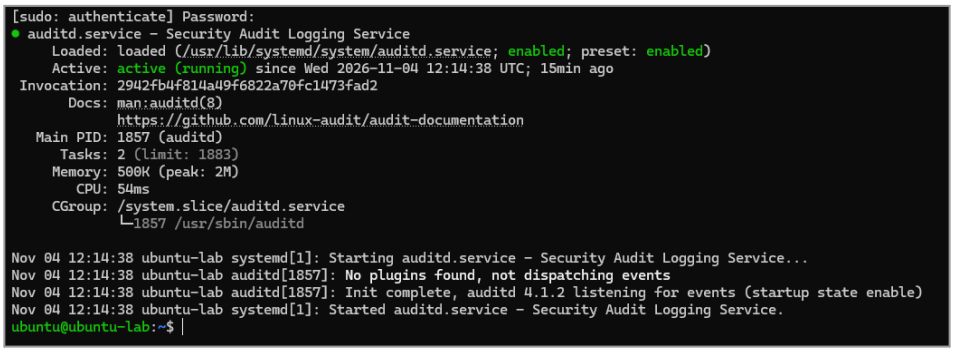


Figure 7: auditd.service active and running

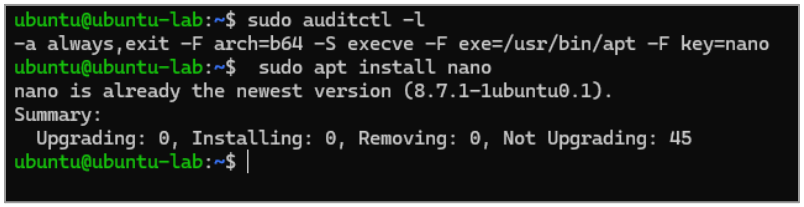


Figure 8: The audit rule with key=nano, and the audited command sudo apt install nano

5.2 Finding the command in the audit log

I searched the log in two ways, by key (sudo ausearch -k nano -i) and by message type (sudo ausearch -m EXECVE -ts recent -i). The -i option turns numbers such as user IDs into readable names.

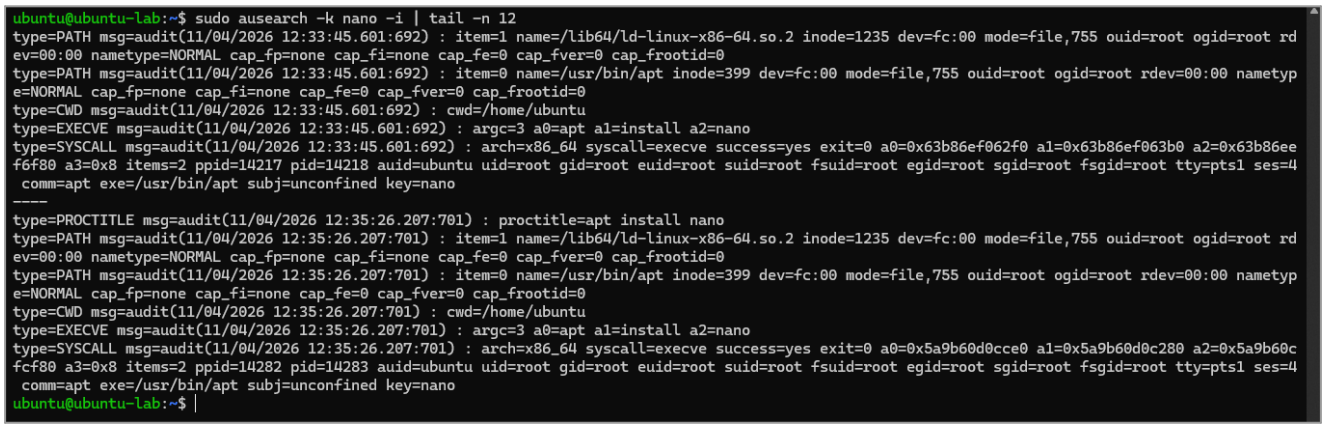


Figure 9: ausearch by key nano: the EXECVE record shows a0=apt, a1=install, a2=nano

```
ubuntu@ubuntu-lab:~$ sudo ausearch -m EXECVE -ts recent -i | tail -n 12
type=PATH msg=audit(11/04/2026 12:33:45.601:692) : item=1 name=/lib64/ld-linux-x86-64.so.2 inode=1235 dev=fc:00 mode=file,755 ouid=root ogid=root rd
ev=00:00 nametype=NORMAL cap_fp=none cap_fi=none cap_fe=0 cap_fver=0 cap_frootid=0
type=PATH msg=audit(11/04/2026 12:33:45.601:692) : item=0 name=/usr/bin/apt inode=399 dev=fc:00 mode=file,755 ouid=root ogid=root rdev=00:00 nametyp
e=NORMAL cap_fp=none cap_fi=none cap_fe=0 cap_fver=0 cap_frootid=0
type=CWD msg=audit(11/04/2026 12:33:45.601:692) : cwd=/home/ubuntu
type=EXECVE msg=audit(11/04/2026 12:33:45.601:692) : argc=3 a0=apt a1=install a2=nano
type=SYSCALL msg=audit(11/04/2026 12:33:45.601:692) : arch=x86_64 syscall=execve success=yes exit=0 a0=0x63b86ef062f0 a1=0x63b86ef063b0 a2=0x63b86ee
f6f80 a3=0x8 items=2 ppid=14217 pid=14218 auid=ubuntu uid=root gid=root euid=root suid=root fsuid=root egid=root sgid=root fsgid=root tty=pts1 ses=4
 comm=apt exe=/usr/bin/apt subj=unconfined key=nano
----
type=PROCTITLE msg=audit(11/04/2026 12:35:26.207:701) : proctitle=apt install nano
type=PATH msg=audit(11/04/2026 12:35:26.207:701) : item=1 name=/lib64/ld-linux-x86-64.so.2 inode=1235 dev=fc:00 mode=file,755 ouid=root ogid=root rd
ev=00:00 nametype=NORMAL cap_fp=none cap_fi=none cap_fe=0 cap_fver=0 cap_frootid=0
type=PATH msg=audit(11/04/2026 12:35:26.207:701) : item=0 name=/usr/bin/apt inode=399 dev=fc:00 mode=file,755 ouid=root ogid=root rdev=00:00 nametyp
e=NORMAL cap_fp=none cap_fi=none cap_fe=0 cap_fver=0 cap_frootid=0
type=CWD msg=audit(11/04/2026 12:35:26.207:701) : cwd=/home/ubuntu
type=EXECVE msg=audit(11/04/2026 12:35:26.207:701) : argc=3 a0=apt a1=install a2=nano
type=SYSCALL msg=audit(11/04/2026 12:35:26.207:701) : arch=x86_64 syscall=execve success=yes exit=0 a0=0x5a9b60d0cce0 a1=0x5a9b60d0c280 a2=0x5a9b60c
fcf80 a3=0x8 items=2 ppid=14282 pid=14283 auid=ubuntu uid=root gid=root euid=root suid=root fsuid=root egid=root sgid=root fsgid=root tty=pts1 ses=4
 comm=apt exe=/usr/bin/apt subj=unconfined key=nano
ubuntu@ubuntu-lab:~$
```

Figure 10: ausearch by message type EXECVE showing the same command

5.3 Important fields in the record

Field	Value in my log	What it tells an analyst
EXECVE a0, a1, a2	apt install nano	The command that was actually run
exe	/usr/bin/apt	The program that ran
auid	ubuntu	The user who originally logged in, which stays the same after sudo
uid	root	The user the command ran as (root, because I used sudo)
tty / ses	pts1 / 4	The terminal and login session, here my SSH session
key	nano	The label from my rule, used to search for the event

Every field is explained in auditd-log-fields.pdf, which is saved with the auditd screenshots.

6. Snort

6.1 The problem: no Snort package

The task says to install Snort with apt. On my Ubuntu 26.04 VM this failed, because the package is not in the repositories for this version. apt answered "Unable to locate package snort" for snort, snort-common and snort-rules-default. Searching the package list and enabling the universe repository did not change this.

```
Reading state information... Done
E: Unable to locate package snort
E: Unable to locate package snort-common
E: Unable to locate package snort-rules-default
ubuntu@ubuntu-lab:~$ snort -V
Command 'snort' not found, did you mean:
  command 'sort' from deb coreutils-from-gnu (0.0.0~ubuntu25)
  command 'sort' from deb coreutils-from-uutils (0.0.0~ubuntu25)
  command 'sort' from deb coreutils-from-busybox (0.0.0~ubuntu25)
  command 'sort' from deb coreutils-from-toybox (0.0.0~ubuntu25)
Try: sudo apt install <deb name>
ubuntu@ubuntu-lab:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP group default qlen 1000
    link/ether 08:00:27:d6:eb:cb brd ff:ff:ff:ff:ff:ff
    altname enx080027d6ebcb
    inet 10.0.2.15/24 metric 100 brd 10.0.2.255 scope global dynamic enp0s3
       valid_lft 84278sec preferred_lft 84278sec
    inet6 fd17:625c:f037:2:a00:27ff:fed6:ebcb/64 scope global dynamic mngtmpaddr noprefixroute
       valid_lft 86178sec preferred_lft 14178sec
    inet6 fe80::a00:27ff:fed6:ebcb/64 scope link proto kernel_ll
       valid_lft forever preferred_lft forever
3: enp0s8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP group default qlen 1000
    link/ether 08:00:27:69:b1:61 brd ff:ff:ff:ff:ff:ff
    altname enx08002769b161
    inet 192.168.56.102/24 metric 100 brd 192.168.56.255 scope global dynamic enp0s8
       valid_lft 578sec preferred_lft 578sec
    inet6 fe80::a00:27ff:fe69:b161/64 scope link proto kernel_ll
       valid_lft forever preferred_lft forever
ubuntu@ubuntu-lab:~$
                                                                                                                                    Show desktop
```

Figure 11: apt cannot locate the snort packages, and snort is not found as a command. The interface list shows enp0s8 with 192.168.56.102

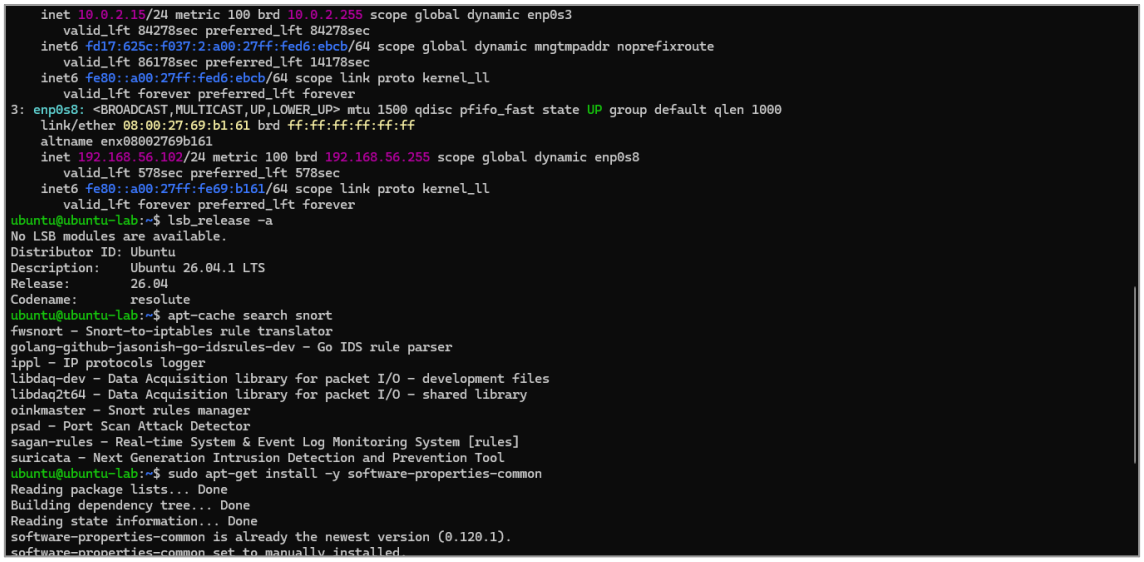


Figure 12: The VM is Ubuntu 26.04 LTS (resolute), and the package search finds no Snort

6.2 The fix: building Snort 3 from source

Snort publishes its source code, so I built Snort 3 myself. I installed the build tools and libraries with apt, built LibDAQ first, then built Snort 3 with cmake. The VM has about 1.6 GB of memory, so I ran make on a single thread to avoid running out of memory. After make install and ldconfig, snort -V showed Snort++ 3.12.2.0.

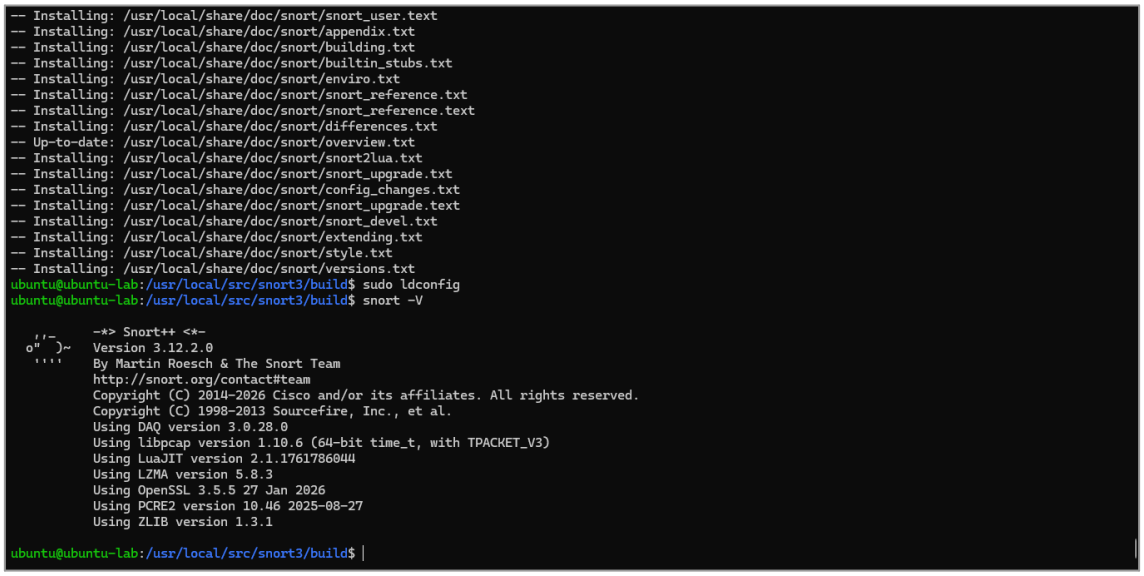


Figure 13: snort -V showing Snort++ version 3.12.2.0 with DAQ 3.0.28.0

6.3 Configuration and starter rules

Snort 3 uses a Lua file instead of snort.conf. In /usr/local/etc/snort/snort.lua I set HOME_NET to 192.168.56.0/24 and EXTERNAL_NET to !$HOME_NET. One of my sed commands failed with "event not found", because bash treats the ! inside double quotes as history expansion. I fixed it by using single quotes around the command. I then created local.rules with the three starter rules:

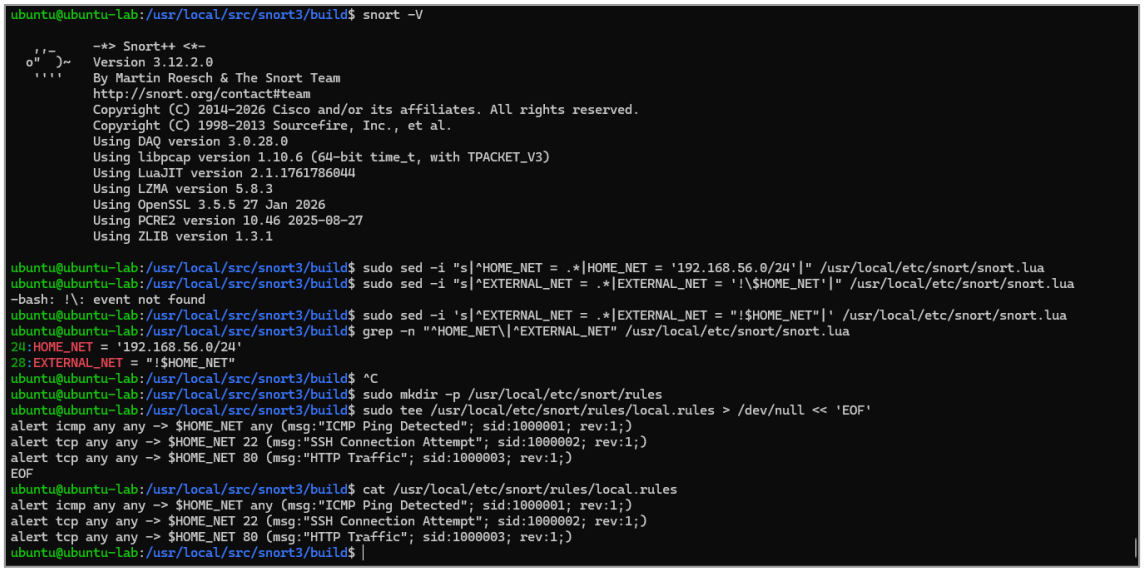


Figure 14: HOME_NET and EXTERNAL_NET set in snort.lua, and the three starter rules in local.rules

I tested the configuration with sudo snort -T (it passed with 0 warnings) and started Snort with:

```
sudo snort -c /usr/local/etc/snort/snort.lua -R /usr/local/etc/snort/rules/local.rules -i enp0s8
-A alert_fast -k none -q
```

From Kali I sent a ping, a web request and an SSH connection to the Ubuntu VM, and all three starter rules raised alerts. The SSH alerts from 192.168.56.1 are caused by my own Windows SSH session to the VM. Later I filtered them out with grep so that they would not flood the screen.

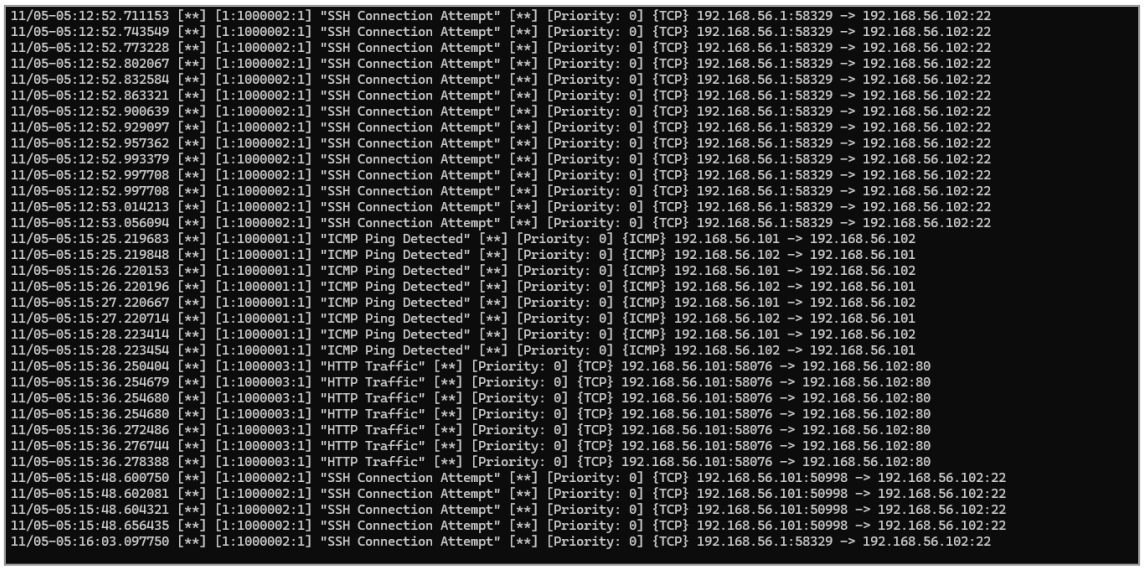


Figure 15: Alerts from the starter rules: SSH Connection Attempt, ICMP Ping Detected and HTTP Traffic

6.4 The T1046 rule: detecting an Nmap SYN scan

I added a fourth rule to detect a SYN scan and tagged it with MITRE ATT&CK T1046, Network Service Discovery:

```
alert tcp any any -> $HOME_NET any (msg:"MITRE T1046 - Nmap SYN Port Scan Detected"; flags:S;
detection_filter:track by_src, count 20, seconds 1; classtype:attempted-recon;
metadata:mitre_technique T1046; reference:url,attack.mitre.org/techniques/T1046/; sid:1000004;
rev:1;)
```

Part of the rule	Meaning
flags:S	Matches TCP packets with the SYN flag set, the first packet of a connection and the signature of a half-open scan

Part of the rule	Meaning
detection_filter ... count 20, seconds 1	Alerts only when one source sends 20 SYN packets within one second, so normal connections do not trigger it
classtype:attempted-recon	Classifies the alert as reconnaissance
metadata:mitre_technique T1046	Tags the rule with the MITRE ATT&CK technique
reference:url,...	Links to the official ATT&CK page for T1046

```
ubuntu@ubuntu-lab:~$ sudo sed -i '/sid:1000004/s/alert tcp \$EXTERNAL_NET any/alert tcp any any/' /usr/local/etc/snort/rules/local.rules
ubuntu@ubuntu-lab:~$ cat /usr/local/etc/snort/rules/local.rules
alert icmp any any -> $HOME_NET any (msg:"ICMP Ping Detected"; sid:1000001; rev:1;)
alert tcp any any -> $HOME_NET 22 (msg:"SSH Connection Attempt"; sid:1000002; rev:1;)
alert tcp any any -> $HOME_NET 80 (msg:"HTTP Traffic"; sid:1000003; rev:1;)
alert tcp any any -> $HOME_NET any (msg:"MITRE T1046 - Nmap SYN Port Scan Detected"; flags:S; detection_filter:track by_src, count 20, seconds 1; cl
asstype:attempted-recon; metadata:mitre_technique T1046; reference:url,attack.mitre.org/techniques/T1046/; sid:1000004; rev:1;)
ubuntu@ubuntu-lab:~$ sudo snort -T -c /usr/local/etc/snort/snort.lua -R /usr/local/etc/snort/rules/local.rules -i enp0s8 2>&1 | tail -5
--------------------------------------------------
pcap DAQ configured to passive.

Snort successfully validated the configuration (with 0 warnings).
o")~   Snort exiting
ubuntu@ubuntu-lab:~$
```

Figure 16: local.rules with all four rules, and the configuration validated with 0 warnings

From Kali I ran sudo nmap -sS 192.168.56.102. Nmap found ports 22 (ssh) and 80 (http) open, and Snort raised the T1046 alert for many ports within a fraction of a second. The Snort times are in UTC, and the scan ran at 11:08 IST, which is 05:38 UTC.

```
┌──(kali㉿kali-lab)-[~]
└─$ sudo nmap -sS 192.168.56.102
Starting Nmap 7.991 ( https://nmap.org ) at 2026-11-05 11:08 +0530
Nmap scan report for 192.168.56.102
Host is up (0.00088s latency).
Not shown: 998 closed tcp ports (reset)
PORT   STATE SERVICE
22/tcp open  ssh
80/tcp open  http
MAC Address:                (Oracle VirtualBox virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 3.31 seconds
```

Figure 17: The Nmap SYN scan from Kali against 192.168.56.102 (MAC address hidden)

```
ubuntu@ubuntu-lab:~$ sudo snort -c /usr/local/etc/snort/snort.lua -R /usr/local/etc/snort/rules/local.rules -i enp0s8 -A alert_fast -k none -q | gre
p --line-buffered -v "192.168.56.1:"
11/05-05:38:15.039683 [**] [1:1000003:1] "HTTP Traffic" [**] [Priority: 0] {TCP} 192.168.56.101:43503 -> 192.168.56.102:80
11/05-05:38:15.040361 [**] [1:1000003:1] "HTTP Traffic" [**] [Priority: 0] {TCP} 192.168.56.101:43503 -> 192.168.56.102:80
11/05-05:38:15.047884 [**] [1:1000004:1] "MITRE T1046 - Nmap SYN Port Scan Detected" [**] [Classification: Attempted Information Leak] [Priority: 2]
 {TCP} 192.168.56.101:43503 -> 192.168.56.102:199
11/05-05:38:15.048686 [**] [1:1000004:1] "MITRE T1046 - Nmap SYN Port Scan Detected" [**] [Classification: Attempted Information Leak] [Priority: 2]
 {TCP} 192.168.56.101:43503 -> 192.168.56.102:445
11/05-05:38:15.049755 [**] [1:1000004:1] "MITRE T1046 - Nmap SYN Port Scan Detected" [**] [Classification: Attempted Information Leak] [Priority: 2]
 {TCP} 192.168.56.101:43503 -> 192.168.56.102:21
11/05-05:38:15.050880 [**] [1:1000004:1] "MITRE T1046 - Nmap SYN Port Scan Detected" [**] [Classification: Attempted Information Leak] [Priority: 2]
 {TCP} 192.168.56.101:43503 -> 192.168.56.102:587
11/05-05:38:15.051429 [**] [1:1000004:1] "MITRE T1046 - Nmap SYN Port Scan Detected" [**] [Classification: Attempted Information Leak] [Priority: 2]
 {TCP} 192.168.56.101:43503 -> 192.168.56.102:110
11/05-05:38:15.051429 [**] [1:1000004:1] "MITRE T1046 - Nmap SYN Port Scan Detected" [**] [Classification: Attempted Information Leak] [Priority: 2]
```

Figure 18: Snort alerts for rule 1000004 (MITRE T1046) at the start of the scan

```
11/05-05:38:15.718541 [**] [1:1000004:1] "MITRE T1046 - Nmap SYN Port Scan Detected" [**] [Classification: Attempted Information Leak] [Priority: 2]
 {TCP} 192.168.56.101:43503 -> 192.168.56.102:9000
11/05-05:38:15.719578 [**] [1:1000004:1] "MITRE T1046 - Nmap SYN Port Scan Detected" [**] [Classification: Attempted Information Leak] [Priority: 2]
 {TCP} 192.168.56.101:43503 -> 192.168.56.102:32768
11/05-05:38:15.719578 [**] [1:1000004:1] "MITRE T1046 - Nmap SYN Port Scan Detected" [**] [Classification: Attempted Information Leak] [Priority: 2]
 {TCP} 192.168.56.101:43503 -> 192.168.56.102:8045
11/05-05:38:15.720555 [**] [1:1000004:1] "MITRE T1046 - Nmap SYN Port Scan Detected" [**] [Classification: Attempted Information Leak] [Priority: 2]
 {TCP} 192.168.56.101:43503 -> 192.168.56.102:2222
11/05-05:38:15.720988 [**] [1:1000004:1] "MITRE T1046 - Nmap SYN Port Scan Detected" [**] [Classification: Attempted Information Leak] [Priority: 2]
 {TCP} 192.168.56.101:43503 -> 192.168.56.102:1078
11/05-05:38:15.720988 [**] [1:1000004:1] "MITRE T1046 - Nmap SYN Port Scan Detected" [**] [Classification: Attempted Information Leak] [Priority: 2]
 {TCP} 192.168.56.101:43503 -> 192.168.56.102:2998
11/05-05:38:15.722003 [**] [1:1000004:1] "MITRE T1046 - Nmap SYN Port Scan Detected" [**] [Classification: Attempted Information Leak] [Priority: 2]
 {TCP} 192.168.56.101:43503 -> 192.168.56.102:56737
11/05-05:38:15.722597 [**] [1:1000004:1] "MITRE T1046 - Nmap SYN Port Scan Detected" [**] [Classification: Attempted Information Leak] [Priority: 2]
 {TCP} 192.168.56.101:43503 -> 192.168.56.102:52869
11/05-05:38:15.722598 [**] [1:1000004:1] "MITRE T1046 - Nmap SYN Port Scan Detected" [**] [Classification: Attempted Information Leak] [Priority: 2]
 {TCP} 192.168.56.101:43503 -> 192.168.56.102:800
11/05-05:38:15.723541 [**] [1:1000004:1] "MITRE T1046 - Nmap SYN Port Scan Detected" [**] [Classification: Attempted Information Leak] [Priority: 2]
```

Figure 19: The last page of alerts from the same scan

6.5 Differences from the task sheet

Snort 3 works differently from the version the task sheet was written for, so I made these changes. They are the only ones, and they are explained here so that the results can be understood.

Task sheet	What I did	Why
Install with apt	Built Snort 3.12.2.0 from source	Ubuntu 26.04 has no Snort package
/etc/snort/snort.conf	/usr/local/etc/snort/snort.lua	Snort 3 uses a Lua configuration file
Rules in /etc/snort/rules/local.rules	/usr/local/etc/snort/rules/local.rules, loaded with -R	Snort 3 loads rules with the -R option
threshold option in rule 4	detection_filter, same count and time	Snort 3 gave "unknown rule keyword: threshold"
Source $EXTERNAL_NET	Source any	Kali is inside HOME_NET, so $EXTERNAL_NET never matched it
Interface eth0 (example)	enp0s8	The real interface name on my VM

7. Cheat sheet

I built a cheat sheet with 19 entries (Event ID or rule, source, what it means, and when to use it). It covers the required entries and a few extra ones. The full version is in the Cheat Sheet folder as event-id-rule-cheat-sheet.pdf. A summary:

Event ID / Rule	Source	What it means
4624	Windows Security	Successful logon
4625	Windows Security	Failed logon
4688	Windows Security	New process created
4720	Windows Security	User account created
4672	Windows Security	Special privileges assigned at logon
1102	Windows Security	Audit log cleared
1	Sysmon	Process Create
3	Sysmon	Network Connection
11	Sysmon	File Create
22	Sysmon	DNS Query
13	Sysmon	Registry Value Set
EXECVE	auditd	Command executed with its arguments
USER_LOGIN	auditd	User login
USER_AUTH	auditd	Authentication attempt
CONFIG_CHANGE	auditd	Audit configuration changed
1000001	Snort	ICMP Ping Detected
1000002	Snort	SSH Connection Attempt
1000003	Snort	HTTP Traffic
1000004	Snort	Nmap SYN Scan Detected (T1046)

8. Rooms and courses completed

TryHackMe rooms

Room	Level	Time	What it covered
Linux Logging for SOC	Easy	45 min	Linux log sources such as auth.log, syslog and the audit log, and how to start a Linux investigation
Intro to Logs	Easy	30 min	What logs are, log types and formats, collection, centralisation, storage and retention
Intro to Log Analysis	Easy	60 min	How to read and investigate logs, command line tools and CyberChef
Snort	Medium	120 min	Snort sniffer, logger and IDS/IPS modes, reading pcap files and writing rules

Each room has its own writeup in the Writeups folder.

LetsDefend courses

Course	Completed	What it covered
How to Investigate a SIEM Alert	5 Nov 2026	Investigating a SIEM alert from detection to the final result
Security Solutions	6 Nov 2026	IDS, IPS, firewall, EDR, antivirus, sandbox, DLP, WAF, proxy and email security
SIEM 101	6 Nov 2026	Log collection, aggregation, parsing, storage and alerting
Incident Management 101	6 Nov 2026	Incident management systems, case naming, playbooks and the analyst workflow
Network Log Analysis	Completed	Examining network log sources during an investigation
Incident Response on Linux	Completed	Responding to incidents on Linux, log files and user startup files

Each course has its own writeup in the Writeups folder.

9. Problems I faced and how I solved them

Problem	What I did
Sysmon install failed with "You need to launch Sysmon as an Administrator"	Reopened PowerShell as Administrator and ran the install again
The test file in the Temp folder was not logged by Sysmon	Learned that the SwiftOnSecurity configuration filters file creation, and used the Downloads folder instead
ausearch -k nano finds nothing without a matching rule	Added an audit rule with the key nano before running the command
apt could not find Snort on Ubuntu 26.04	Built Snort 3 from source, using a single-thread build because the VM has little memory
A sed command with ! failed with "event not found"	Used single quotes so that bash did not treat ! as history expansion
Rule 4 failed with "unknown rule keyword: threshold"	Used detection_filter, which does the same job in Snort 3
The scan raised no alerts at first	Found that Kali is inside HOME_NET, so I changed the rule source from $EXTERNAL_NET to any
My own SSH session filled the screen with SSH alerts	Filtered out the alerts from 192.168.56.1 with grep

10. What I learned

Endpoint logs give much more context than standard operating system logs. Sysmon showed me the full command line, the process behind a network connection and where files were created, and auditd showed me the command that ran and the original user behind it, even after sudo. I also learned that a detection setup is only as good as its configuration, because the Temp folder file was never logged until I understood how the configuration filters events.

Snort taught me how a rule is built and why testing matters. Working through the Snort 3 differences made me read error messages carefully and understand each option, instead of copying the rule as it was. The LetsDefend and TryHackMe work gave me the bigger picture of how these logs reach a SIEM and how an analyst investigates an alert from start to finish.

Skills practised this week

• Installing and configuring Sysmon and finding events in Event Viewer

• Writing audit rules and searching the audit log with ausearch

• Building software from source, and configuring and testing Snort 3

• Writing a Snort detection rule and mapping it to a MITRE ATT&CK technique

• Documenting work clearly, with screenshots as evidence

11. Conclusion

This week I completed all the hands-on tasks, the theory notes, the TryHackMe rooms and the LetsDefend courses. I also shared my learning on LinkedIn with two posts, one on why endpoint logging is more valuable than standard OS logs, and one on my weekly experience, tagging DG Interns Hub. I am thankful to DG Interns Hub for the structured tasks, which helped me move from reading about these tools to using them.

Appendix A: local.rules

The rules file that I used with Snort 3 (saved in the Snort Rules folder):

```
alert icmp any any -> $HOME_NET any (msg:"ICMP Ping Detected"; sid:1000001; rev:1;)

alert tcp any any -> $HOME_NET 22 (msg:"SSH Connection Attempt"; sid:1000002; rev:1;)

alert tcp any any -> $HOME_NET 80 (msg:"HTTP Traffic"; sid:1000003; rev:1;)

alert tcp any any -> $HOME_NET any (msg:"MITRE T1046 - Nmap SYN Port Scan Detected"; flags:S;
detection_filter:track by_src, count 20, seconds 1; classtype:attempted-recon;
metadata:mitre_technique T1046; reference:url,attack.mitre.org/techniques/T1046/; sid:1000004;
rev:1;)
```

Appendix B: Week 3 folder structure

```
DG Interns Hub / Week 3
```

Folder or file	Contents
Notes	The 10 theory answers, one file for each
Writeups	TryHackMe and LetsDefend writeups, one file for each
Findings	Screenshots: Sysmon events, auditd logs, Snort setup and alerts, with the explanation documents
Cheat Sheet	Event ID / Rule cheat sheet
Snort Rules	local.rules

Folder or file	Contents
PDF report, presentation, README	This report, the presentation and the README, directly inside Week 3